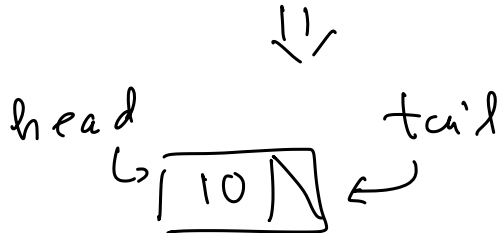
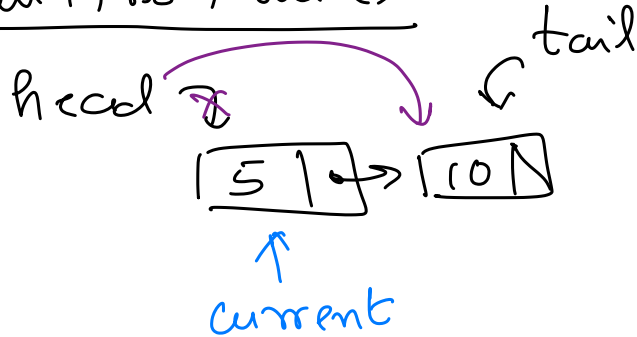


Delete first Node

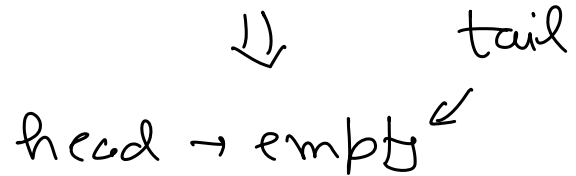
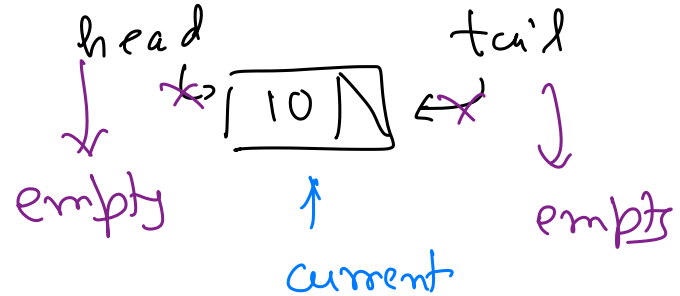
Delete First Node()

head $\rightarrow$ empty $\leftarrow$ tail $\Leftarrow$ Stop if list is empty.

Delete First Node()



Delete First Node()



DeleteFirstNode()

- if list is empty then → if (isEmpty()) // if (head == null)
 - Stop → return;
- Set temp to head. → Node temp = head;
- Set head to head's next. → head = head.next;
- if list is empty then
 - Set tail to head. → if (isEmpty()) // if (head == null)
 - Stop. → tail = head; // tail = null;
- return temp.data;

List as ADT

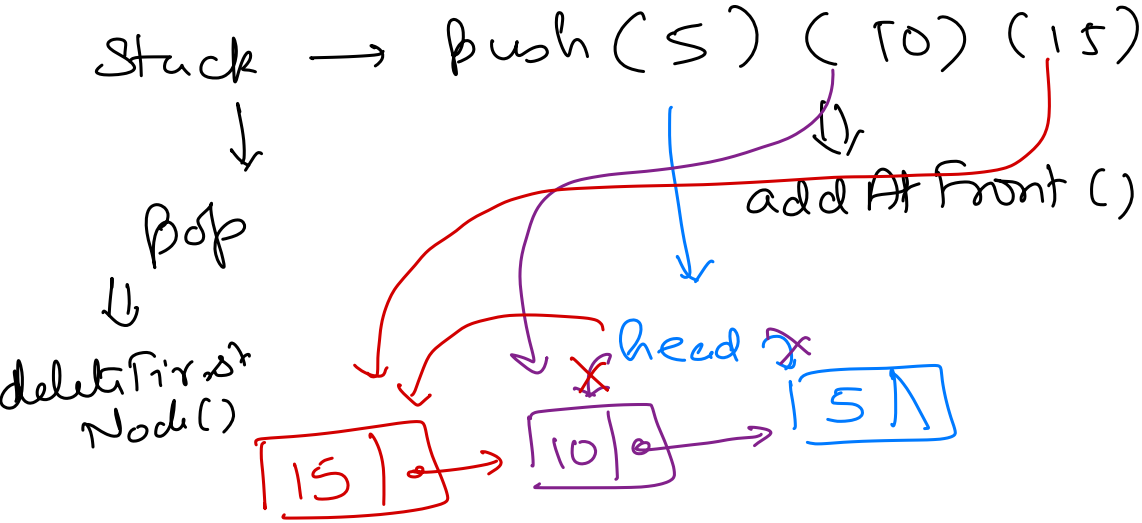
... interface List {

void addAtFront(int element);

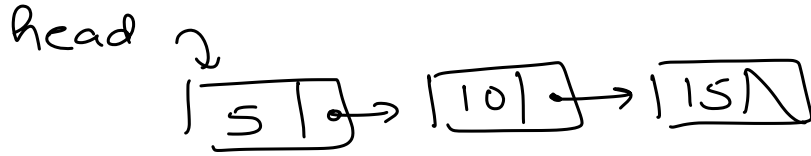
void addAtRear(...);

boolean isEmpty(); ... }

Implement Stack using linked list

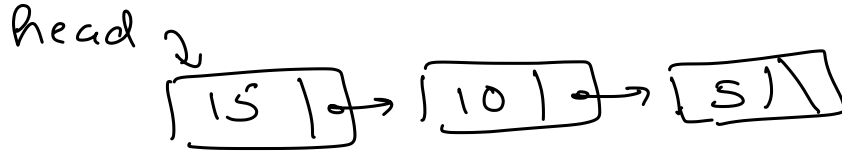


① Reverse a singly list.



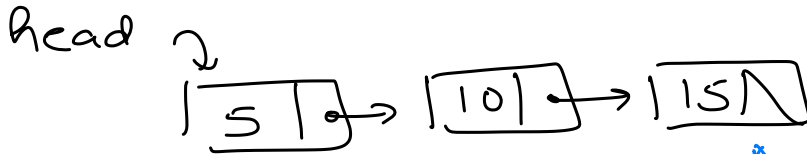
→ Using stack

⇓ Reverse



→ Use 2/3
pointers

② Find 2nd last node of list.



→ use stack

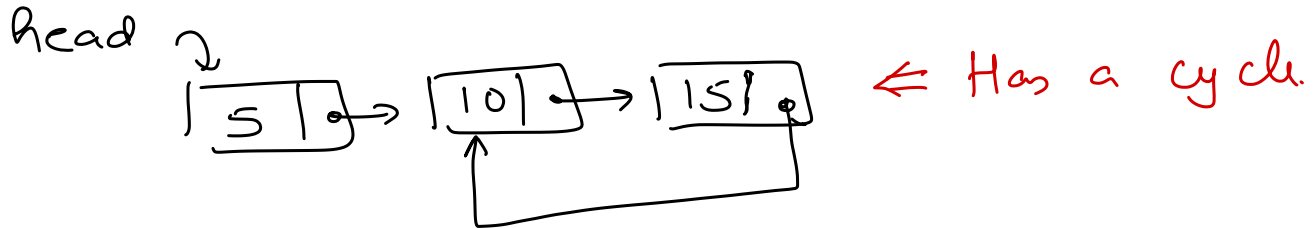
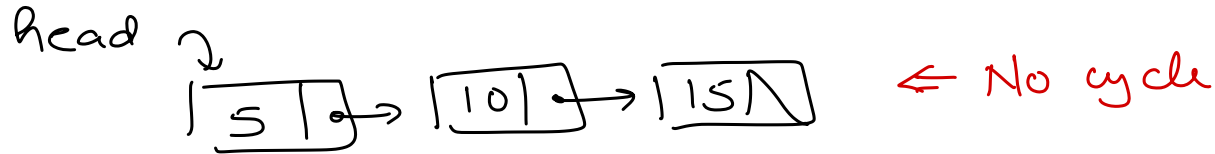
↑
Previous

↑
Current

→ Two pointer

③ Find k^{th} last node. $\rightarrow$ Use stack
 $\rightarrow$ use 2 pointers, the two pointers will move k nodes apart.

④ Check if list has a cycle.



→ Keep track of nodes already visited.

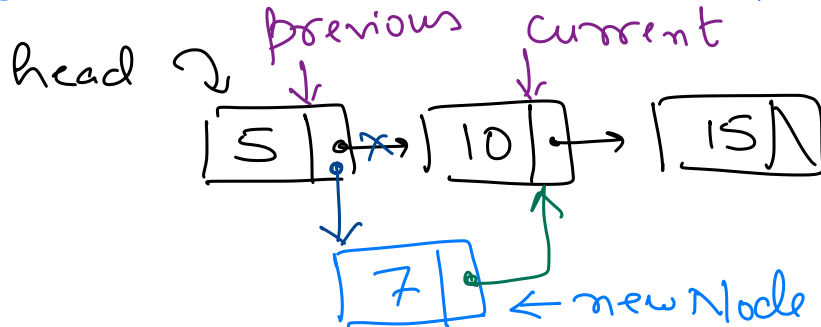
→ Use 2 pointers (hare & tortoise)

Insert in a list

→ add a element at a specific location.

→ add element after / before a specific value.

→ add element to a sorted list.



insert (7)

① Create a new Node.

② Store element in new Node.

③ Traverse list to find current node (node before which new Node will be added).

for elements { first node having
stored in value greater than
ascending order element to be inserted.

→ Set current to head (first node)

→ Set previous to empty.

→ while (current is not empty)

 → if (current.data > element)

→ end traversal.
→ Set previous to current
→ Set current to its next node.

- ④ Add new Node between previous and current.
→ Set new Node as next of previous.
→ Set current as next of new Node.

Special / corner cases

① Empty list

② Inserting largest value.

③ Inserting smallest value.

Insert(element)

- Make space for new element, say newNode. $\rightarrow$ `Node newNode = new Node();`
- Store element in newNode's data. $\rightarrow$ `newNode.data = element;`
- Set newNode's next to empty. $\rightarrow$ `newNode.next = null;`
- if list is empty then $\rightarrow$ `if (head == null) {`
 - Make newNode as first (and only) node of list. $\rightarrow$ `head = newNode;`
 - Stop $\rightarrow$ `return;``}`

// List is not empty

// $\Rightarrow$ Find first node having data greater than newNode's data.

- Set current to first node. $\rightarrow$ `Node current = head;`
- Set previous to empty. $\rightarrow$ `Node previous = null;`
- while (current is not empty) do $\rightarrow$ `while (current != null) {`
 - if (current node's data > newNode's data) then $\rightarrow$ `if (current.data > newNode.data) {`
 - // Found the node $\rightarrow$ `break;`
 - End the traversal.
 - Set previous to current. $\rightarrow$ `previous = current;`
 - Move current to current's next node. $\rightarrow$ `current = current.next;`

- if (previous is empty) then // newNode's data is smallest
- // Add newNode as first node.
- Set newNode's next to first node.
- Make newNode as first node.
- Stop.

```

}
if (previous == empty)
// if (current == head)
newNode.next = head;
head = newNode;
return;
}

```

// Add newNode between previous and current

- Set newNode as next of previous.
- Set current as next of newNode.
- Stop.

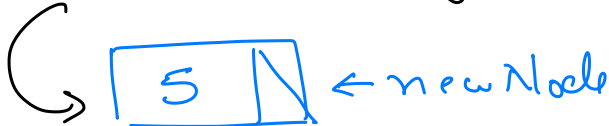
```

previous.next = newNode;
newNode.next = current;

```

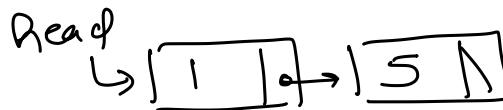
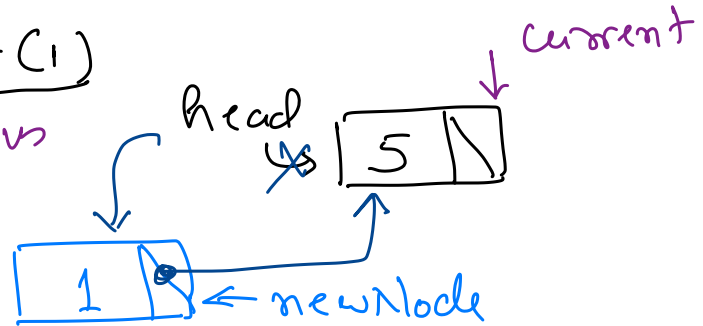
insert (5)

head ~~is~~ empty

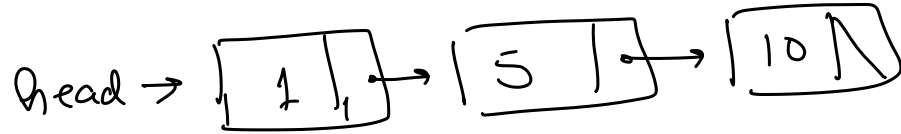
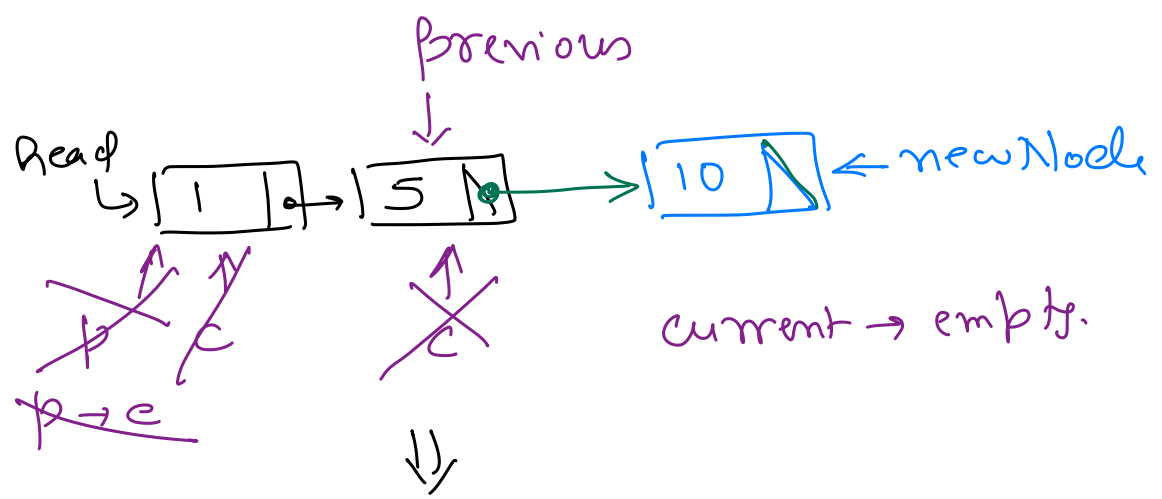


insert (1)

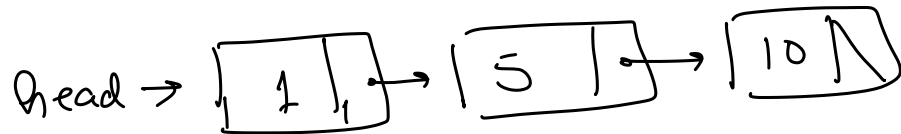
previous
↓
empty



insert (10)



insert (7)



Exercise: Implement insert for a sorted singly linked list that uses head and tail pointers.

Insert in a singly list using only one pointer. Using only current and no previous.

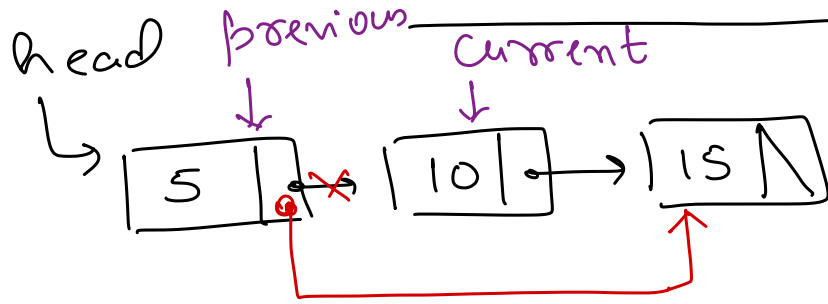
① Do not let current become empty.



Stop traversal at last node.

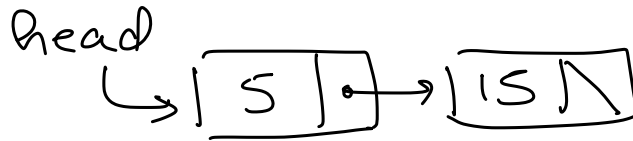
while (current.next != null)

Delete a node



Set previous node's next to
current node's next.

⇓



delete (10)

Special cases

- ① Empty list
- ② Element not present
- ③ List has only one node
- ④ Delete first node.

Delete(element)

- Set current to first node (head)
- Set previous to empty.
- while (current is not empty) do
 - if (current node's data = element) then
 - End the traversal. // Element found
 - Set previous to current node.
 - Move current to current node's next node.
- if (current is empty) then // Element not present in list
 - Stop
- if (current node is first node of list) then // Deleting first node
 - Move head to head's next node.
 - Mark current node as free.
 - Stop.
- Set previous node's next to current node's next.
- Mark current node as free. *← Not needed in JAVA.*
- Stop.